```
import crypto from 'node:crypto';
import { one } from '../db.js';

// A package is not installable until it is signed. The signature covers the
// manifest hash, so changing what a package declares invalidates it.
const KEY = process.env.PACKAGE_SIGNING_KEY ?? 'development-signing-key';

export async function sign({ packageKey, version, contentHash, signer, trustTier
  const signature = crypto.createHmac('sha256', KEY)
    .update(`${packageKey}@${version}:${contentHash}`).digest('hex');

  return one(
    `insert into package_signature (package_key, version, content_hash, signature
     values ($1,$2,$3,$4,$5,$6)
     on conflict (package_key, version) do update set content_hash = excluded.con
       signature = excluded.signature, signer = excluded.signer, trust_tier = exc
     returning *`,
    [packageKey, version, contentHash, signature, signer, trustTier]
  );
}

export async function verify({ packageKey, version, contentHash }) {
  const row = await one(
    `select * from package_signature where package_key = $1 and version = $2`, [p
  );
  if (!row) return { ok: false, reason: 'unsigned' };

  const expected = crypto.createHmac('sha256', KEY)
    .update(`${packageKey}@${version}:${row.content_hash}`).digest('hex');
  if (row.signature !== expected) return { ok: false, reason: 'signature does not
  if (contentHash && row.content_hash !== contentHash) {
    return { ok: false, reason: 'manifest changed since it was signed' };
  }
  return { ok: true, signer: row.signer, trustTier: row.trust_tier };
}
```